

Procesamiento Masivo de Datos

OpenMP II y MPI I

Alex Di Genova

Universidad de O'Higgins

OpenMP: formar un equipo y repartir trabajo

Repaso · Memoria compartida y modelo fork-join

- API para C, C++ y Fortran: directivas, rutinas y variables de entorno.

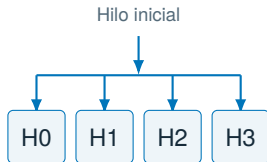
Hilo inicial



OpenMP: formar un equipo y repartir trabajo

Repaso · Memoria compartida y modelo fork-join

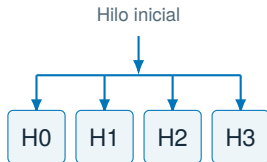
- API para C, C++ y Fortran: directivas, rutinas y variables de entorno.
- `parallel` forma un equipo; cada hilo ejecuta el bloque.



OpenMP: formar un equipo y repartir trabajo

Repaso · Memoria compartida y modelo fork-join

- API para C, C++ y Fortran: directivas, rutinas y variables de entorno.
- `parallel` forma un equipo; cada hilo ejecuta el bloque.
- `for` reparte iteraciones entre los hilos del equipo.



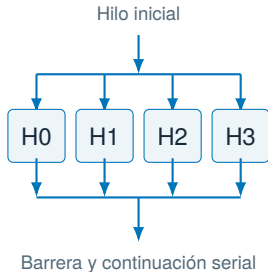
```
#pragma omp parallel
{
    #pragma omp for
    for (int i = 0; i < n; ++i) procesar(i);
}
```

OpenMP: formar un equipo y repartir trabajo

Repaso · Memoria compartida y modelo fork-join

- API para C, C++ y Fortran: directivas, rutinas y variables de entorno.
- `parallel` forma un equipo; cada hilo ejecuta el bloque.
- `for` reparte iteraciones entre los hilos del equipo.
- Al terminar la región paralela hay una barrera implícita.

```
#pragma omp parallel
{
    #pragma omp for
    for (int i = 0; i < n; ++i) procesar(i);
}
```



Antes de paralelizar: ¿quién usa cada variable?

Repaso · Alcance de datos

Cláusula / regla	Efecto	Uso típico
<code>shared(a)</code>	Todos acceden al mismo objeto.	Arreglo de entrada de solo lectura.

Antes de paralelizar: ¿quién usa cada variable?

Repaso · Alcance de datos

Cláusula / regla	Efecto	Uso típico
<code>shared(a)</code>	Todos acceden al mismo objeto.	Arreglo de entrada de solo lectura.
<code>private(x)</code>	Una copia por hilo; no hereda el valor inicial.	Temporal que se asigna antes de leer.

Antes de paralelizar: ¿quién usa cada variable?

Repaso · Alcance de datos

Cláusula / regla	Efecto	Uso típico
<code>shared(a)</code>	Todos acceden al mismo objeto.	Arreglo de entrada de solo lectura.
<code>private(x)</code>	Una copia por hilo; no hereda el valor inicial.	Temporal que se asigna antes de leer.
<code>firstprivate(x)</code>	Una copia inicializada con el valor previo.	Parámetros de una tarea.

Antes de paralelizar: ¿quién usa cada variable?

Repaso · Alcance de datos

Cláusula / regla	Efecto	Uso típico
<code>shared(a)</code>	Todos acceden al mismo objeto.	Arreglo de entrada de solo lectura.
<code>private(x)</code>	Una copia por hilo; no hereda el valor inicial.	Temporal que se asigna antes de leer.
<code>firstprivate(x)</code>	Una copia inicializada con el valor previo.	Parámetros de una tarea.
<code>reduction(+:s)</code>	Acumuladores privados que se combinan.	Suma, conteo o producto escalar.

Antes de paralelizar: ¿quién usa cada variable?

Repaso · Alcance de datos

Cláusula / regla	Efecto	Uso típico
<code>shared(a)</code>	Todos acceden al mismo objeto.	Arreglo de entrada de solo lectura.
<code>private(x)</code>	Una copia por hilo; no hereda el valor inicial.	Temporal que se asigna antes de leer.
<code>firstprivate(x)</code>	Una copia inicializada con el valor previo.	Parámetros de una tarea.
<code>reduction(+:s)</code>	Acumuladores privados que se combinan.	Suma, conteo o producto escalar.
<code>default(none)</code>	Exige atributos explícitos, salvo los predeterminados.	Revisar el alcance de datos.

Antes de paralelizar: ¿quién usa cada variable?

Repaso · Alcance de datos

Cláusula / regla	Efecto	Uso típico
<code>shared(a)</code>	Todos acceden al mismo objeto.	Arreglo de entrada de solo lectura.
<code>private(x)</code>	Una copia por hilo; no hereda el valor inicial.	Temporal que se asigna antes de leer.
<code>firstprivate(x)</code>	Una copia inicializada con el valor previo.	Parámetros de una tarea.
<code>reduction(+:s)</code>	Acumuladores privados que se combinan.	Suma, conteo o producto escalar.
<code>default(none)</code>	Exige atributos explícitos, salvo los predeterminados.	Revisar el alcance de datos.

Dos reglas útiles

El índice del `omp for` y las variables automáticas del bloque paralelo son privados. Copiar un puntero no copia el arreglo al que apunta.

Ejemplo simple 2: sumar dos vectores

Una iteración escribe una posición distinta

```
#pragma omp parallel for default(none) shared(a,b,c) schedule(static)
for (int i = 0; i < N; ++i) {
    c[i] = a[i] + b[i];
}
```

Programa completo: `examples/02_vectores.c`

Ejemplo simple 2: sumar dos vectores

Una iteración escribe una posición distinta

```
#pragma omp parallel for default(none) shared(a,b,c) schedule(static)
for (int i = 0; i < N; ++i) {
    c[i] = a[i] + b[i];
}
```

Datos de entrada

$N = 12$, $a_i = i$, $b_i = 2i$.

Cada iteración calcula $c_i = a_i + b_i = 3i$.

Programa completo: `examples/02_vectores.c`

Ejemplo simple 2: sumar dos vectores

Una iteración escribe una posición distinta

```
#pragma omp parallel for default(none) shared(a,b,c) schedule(static)
for (int i = 0; i < N; ++i) {
    c[i] = a[i] + b[i];
}
```

Datos de entrada

$N = 12$, $a_i = i$, $b_i = 2i$.

Cada iteración calcula $c_i = a_i + b_i = 3i$.

Por qué es correcto

a y b solo se leen; cada posición de c tiene un único escritor. Compartir un arreglo no implica necesariamente una carrera.

Programa completo: `examples/02_vectores.c`

Ejemplo simple 2: sumar dos vectores

Una iteración escribe una posición distinta

```
#pragma omp parallel for default(none) shared(a,b,c) schedule(static)
for (int i = 0; i < N; ++i) {
    c[i] = a[i] + b[i];
}
```

Datos de entrada

$N = 12$, $a_i = i$, $b_i = 2i$.

Cada iteración calcula $c_i = a_i + b_i = 3i$.

Resultado: 0 3 6 9 12 15 18 21 24 27 30 33.

Programa completo: `examples/02_vectores.c`

Por qué es correcto

a y b solo se leen; cada posición de c tiene un único escritor. Compartir un arreglo no implica necesariamente una carrera.

¿Qué falla si todos actualizan la misma suma?

Ejemplo incorrecto para discutir · No ejecutar como solución

```
int suma = 0;
#pragma omp parallel for
    shared(a,suma)
for (int i = 0; i < n; ++i) {
    suma += a[i]; // CARRERA
}
```

¿Qué falla si todos actualizan la misma suma?

Ejemplo incorrecto para discutir · No ejecutar como solución

```
int suma = 0;
#pragma omp parallel for
    shared(a,suma)
for (int i = 0; i < n; ++i) {
    suma += a[i]; // CARRERA
}
```

La lectura, la suma y la escritura no forman una actualización atómica.

Paso	Hilo 0	Hilo 1
1	Lee 0	Lee 0

¿Qué falla si todos actualizan la misma suma?

Ejemplo incorrecto para discutir · No ejecutar como solución

```
int suma = 0;
#pragma omp parallel for
    shared(a,suma)
for (int i = 0; i < n; ++i) {
    suma += a[i];  // CARRERA
}
```

La lectura, la suma y la escritura no forman una actualización atómica.

Paso	Hilo 0	Hilo 1
1	Lee 0	Lee 0
2	Calcula 0 + 1	Calcula 0 + 2

¿Qué falla si todos actualizan la misma suma?

Ejemplo incorrecto para discutir · No ejecutar como solución

```
int suma = 0;
#pragma omp parallel for
    shared(a,suma)
for (int i = 0; i < n; ++i) {
    suma += a[i]; // CARRERA
}
```

La lectura, la suma y la escritura no forman una actualización atómica.

Paso	Hilo 0	Hilo 1
1	Lee 0	Lee 0
2	Calcula 0 + 1	Calcula 0 + 2
3	Escribe 1	Escribe 2

¿Qué falla si todos actualizan la misma suma?

Ejemplo incorrecto para discutir · No ejecutar como solución

```
int suma = 0;
#pragma omp parallel for
    shared(a,suma)
for (int i = 0; i < n; ++i) {
    suma += a[i]; // CARRERA
}
```

La lectura, la suma y la escritura no forman una actualización atómica.

Paso	Hilo 0	Hilo 1
1	Lee 0	Lee 0
2	Calcula 0 + 1	Calcula 0 + 2
3	Escribe 1	Escribe 2

Actualización perdida

Esta intercalación ilustra cómo se pierde una contribución. La carrera produce comportamiento indefinido en C; no existe una salida fiable.

Ejemplo simple 3: sumar con reduction

Cada hilo acumula; OpenMP combina

```
int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
int n = 10;
int suma = 0;
#pragma omp parallel for default(none) shared(a,n) reduction(+:suma)
for (int i = 0; i < n; ++i) {
    suma += a[i];
}
printf("Suma = %d\n", suma);
```

Ejemplo simple 3: sumar con reduction

Cada hilo acumula; OpenMP combina

```
int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
int n = 10;
int suma = 0;
#pragma omp parallel for default(none) shared(a,n) reduction(+:suma)
for (int i = 0; i < n; ++i) {
    suma += a[i];
}
printf("Suma = %d\n", suma);
```

Resultado esperado: 55

Las copias privadas de `suma` comienzan en cero. Al finalizar, sus contribuciones se combinan con el valor original mediante `+`. Inicializamos la suma original en cero.

Ejemplo simple 3: sumar con reduction

Cada hilo acumula; OpenMP combina

```
int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
int n = 10;
int suma = 0;
#pragma omp parallel for default(none) shared(a,n) reduction(+:suma)
for (int i = 0; i < n; ++i) {
    suma += a[i];
}
printf("Suma = %d\n", suma);
```

Resultado esperado: 55

Las copias privadas de `suma` comienzan en cero. Al finalizar, sus contribuciones se combinan con el valor original mediante `+`. Inicializamos la suma original en cero.

Cambio propuesto: reemplazar la suma por un producto escalar $\sum_i a_i b_i$.

Ejemplo simple 4: promedio de un arreglo

Funciona aunque N no sea múltiplo del número de hilos

```
double suma = 0.0;
#pragma omp parallel for default(none) shared(a,n) reduction(+:suma)
for (long i = 0; i < n; ++i) {
    suma += a[i];
}
double promedio = suma / (double)n;
```

Ejemplo simple 4: promedio de un arreglo

Funciona aunque N no sea múltiplo del número de hilos

```
double suma = 0.0;
#pragma omp parallel for default(none) shared(a,n) reduction(+:suma)
for (long i = 0; i < n; ++i) {
    suma += a[i];
}
double promedio = suma / (double)n;
```

```
OMP_NUM_THREADS=4 ./build/examples/04_promedio 10003
# N=10003 promedio=5001.000000 esperado=5001.000000
```

Ejemplo simple 4: promedio de un arreglo

Funciona aunque N no sea múltiplo del número de hilos

```
double suma = 0.0;
#pragma omp parallel for default(none) shared(a,n) reduction(+:suma)
for (long i = 0; i < n; ++i) {
    suma += a[i];
}
double promedio = suma / (double)n;
```

```
OMP_NUM_THREADS=4 ./build/examples/04_promedio 10003
# N=10003 promedio=5001.000000 esperado=5001.000000
```

Verificación independiente

El arreglo contiene $0, 1, \dots, N - 1$; su promedio es $(N - 1)/2$. Probar también $N = 1$, $N = 3$ y $N = 10$. El programa rechaza $N = 0$.

Elegir la sincronización necesaria

Correctitud primero; después, costo

Construcción	Qué garantiza	Ejemplo de uso
<code>reduction</code>	Acumulación privada y combinación.	Suma de un vector.

Elegir la sincronización necesaria

Correctitud primero; después, costo

Construcción	Qué garantiza	Ejemplo de uso
<code>reduction</code>	Acumulación privada y combinación.	Suma de un vector.
<code>atomic update</code>	Actualización atómica de una ubicación compatible.	<code>contador++;</code>

Elegir la sincronización necesaria

Correctitud primero; después, costo

Construcción	Qué garantiza	Ejemplo de uso
<code>reduction</code>	Acumulación privada y combinación.	Suma de un vector.
<code>atomic update</code>	Actualización atómica de una ubicación compatible.	<code>contador++;</code>
<code>critical(nombre)</code>	Exclusión mutua para un bloque con ese nombre.	Fusionar un histograma local.

Elegir la sincronización necesaria

Correctitud primero; después, costo

Construcción	Qué garantiza	Ejemplo de uso
<code>reduction</code>	Acumulación privada y combinación.	Suma de un vector.
<code>atomic update</code>	Actualización atómica de una ubicación compatible.	<code>contador++;</code>
<code>critical(nombre)</code>	Exclusión mutua para un bloque con ese nombre.	Fusionar un histograma local.
<code>barrier</code>	El equipo espera; todo hilo debe alcanzarla.	Leer datos producidos por otros hilos.

Elegir la sincronización necesaria

Correctitud primero; después, costo

Construcción	Qué garantiza	Ejemplo de uso
<code>reduction</code>	Acumulación privada y combinación.	Suma de un vector.
<code>atomic update</code>	Actualización atómica de una ubicación compatible.	<code>contador++;</code>
<code>critical(nombre)</code>	Exclusión mutua para un bloque con ese nombre.	Fusionar un histograma local.
<code>barrier</code>	El equipo espera; todo hilo debe alcanzarla.	Leer datos producidos por otros hilos.
<code>taskwait</code>	Espera las tareas hijas previas de la tarea actual.	Finalizar una etapa recursiva.

Elegir la sincronización necesaria

Correctitud primero; después, costo

Construcción	Qué garantiza	Ejemplo de uso
<code>reduction</code>	Acumulación privada y combinación.	Suma de un vector.
<code>atomic update</code>	Actualización atómica de una ubicación compatible.	<code>contador++</code> ;
<code>critical(nombre)</code>	Exclusión mutua para un bloque con ese nombre.	Fusionar un histograma local.
<code>barrier</code>	El equipo espera; todo hilo debe alcanzarla.	Leer datos producidos por otros hilos.
<code>taskwait</code>	Espera las tareas hijas previas de la tarea actual.	Finalizar una etapa recursiva.

Sobre `nowait`

`for`, `sections` y `single` tienen barrera final salvo `nowait`. Quitarla exige que el siguiente paso no dependa de resultados pendientes.

Ejemplo intermedio 1: histograma de un texto

De muchas actualizaciones compartidas a contadores locales

1. Repartir las posiciones del texto con omp for.

Entrada: banana bandana

Byte	espacio	a	b	d	n
------	---------	---	---	---	---

Cuenta		1	6	2	1	3
--------	--	---	---	---	---	---

Total: 13 bytes.

Ejemplo intermedio 1: histograma de un texto

De muchas actualizaciones compartidas a contadores locales

1. Repartir las posiciones del texto con `omp for`.
2. Cada hilo acumula en su propio arreglo de 256 contadores.

Entrada: banana bandana

Byte	espacio	a	b	d	n
------	---------	---	---	---	---

Cuenta		1	6	2	1	3
--------	--	---	---	---	---	---

Total: 13 bytes.

Ejemplo intermedio 1: histograma de un texto

De muchas actualizaciones compartidas a contadores locales

1. Repartir las posiciones del texto con `omp for`.
2. Cada hilo acumula en su propio arreglo de 256 contadores.
3. Fusionar cada arreglo local en una región `critical`.

Entrada: banana bandana

Byte	espacio	a	b	d	n	
Cuenta		1	6	2	1	3

Total: 13 bytes.

Ejemplo intermedio 1: histograma de un texto

De muchas actualizaciones compartidas a contadores locales

1. Repartir las posiciones del texto con `omp for`.
2. Cada hilo acumula en su propio arreglo de 256 contadores.
3. Fusionar cada arreglo local en una región `critical`.

Entrada: banana bandana

Byte	espacio	a	b	d	n	
Cuenta		1	6	2	1	3

Total: 13 bytes.

Qué estamos contando

256 valores posibles de un byte de 8 bits. ASCII tiene 128 códigos. En UTF-8 una letra como «á» ocupa varios bytes.

Se recorre el intervalo $[0,n)$: no se copian mitades con `strncpy` ni se depende de terminadores intermedios.

Histograma: privatizar y fusionar

Intermedio · Arreglo local por hilo

```
#pragma omp parallel default(none) shared(texto,n,hist)
{
    unsigned long local[256] = {0};
    #pragma omp for schedule(static)
    for (size_t i = 0; i < n; ++i) {
        unsigned char b = (unsigned char)texto[i];
        local[b]++;
    }
    #pragma omp critical(fusion_histograma)
    {
        for (int b = 0; b < 256; ++b)
            hist[b] += local[b];
    }
}
```

Programa completo: `examples/06_histograma.c`

Histograma: privatizar y fusionar

Intermedio · Arreglo local por hilo

```
#pragma omp parallel default(none) shared(texto,n,hist)
{
    unsigned long local[256] = {0};
    #pragma omp for schedule(static)
    for (size_t i = 0; i < n; ++i) {
        unsigned char b = (unsigned char)texto[i];
        local[b]++;
    }
    #pragma omp critical(fusion_histograma)
    {
        for (int b = 0; b < 256; ++b)
            hist[b] += local[b];
    }
}
```

Privado: local, i y b.

Programa completo: `examples/06_histograma.c`

Histograma: privatizar y fusionar

Intermedio · Arreglo local por hilo

```
#pragma omp parallel default(none) shared(texto,n,hist)
{
    unsigned long local[256] = {0};
    #pragma omp for schedule(static)
    for (size_t i = 0; i < n; ++i) {
        unsigned char b = (unsigned char)texto[i];
        local[b]++;
    }
    #pragma omp critical(fusion_histograma)
    {
        for (int b = 0; b < 256; ++b)
            hist[b] += local[b];
    }
}
```

Privado: local, i y b.

Compartido: texto, n y hist.

Programa completo: `examples/06_histograma.c`

Histograma: privatizar y fusionar

Intermedio · Arreglo local por hilo

```
#pragma omp parallel default(none) shared(texto,n,hist)
{
    unsigned long local[256] = {0};
    #pragma omp for schedule(static)
    for (size_t i = 0; i < n; ++i) {
        unsigned char b = (unsigned char)texto[i];
        local[b]++;
    }
    #pragma omp critical(fusion_histograma)
    {
        for (int b = 0; b < 256; ++b)
            hist[b] += local[b];
    }
}
```

Programa completo: `examples/06_histograma.c`

Privado: local, i y b.

Compartido: texto, n y hist.

El cast a unsigned char evita índices negativos.

Histograma: privatizar y fusionar

Intermedio · Arreglo local por hilo

```
#pragma omp parallel default(none) shared(texto,n,hist)
{
    unsigned long local[256] = {0};
    #pragma omp for schedule(static)
    for (size_t i = 0; i < n; ++i) {
        unsigned char b = (unsigned char)texto[i];
        local[b]++;
    }
    #pragma omp critical(fusion_histograma)
    {
        for (int b = 0; b < 256; ++b)
            hist[b] += local[b];
    }
}
```

Programa completo: `examples/06_histograma.c`

Privado: local, i y b.

Compartido: texto, n y hist.

El cast a unsigned char evita índices negativos.

Se entra a critical una vez por hilo, en lugar de una vez por byte.

Ejemplo intermedio 3: quicksort

Dividir para conquistar · Dos subarreglos independientes

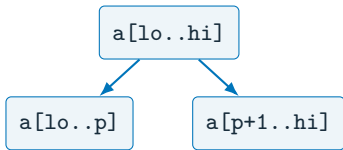
1. Elegir un valor pivote.

`a[lo..hi]`

Ejemplo intermedio 3: quicksort

Dividir para conquistar · Dos subarreglos independientes

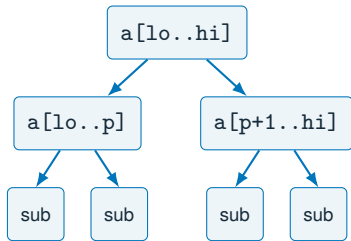
1. Elegir un valor pivote.
2. Particionar el intervalo de modo que los valores pequeños queden a la izquierda.



Ejemplo intermedio 3: quicksort

Dividir para conquistar · Dos subarreglos independientes

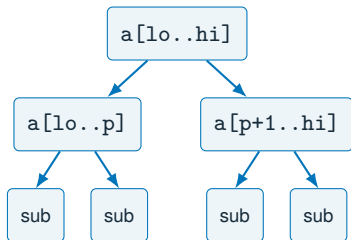
1. Elegir un valor pivote.
2. Particionar el intervalo de modo que los valores pequeños queden a la izquierda.
3. Ordenar recursivamente ambos intervalos.



Ejemplo intermedio 3: quicksort

Dividir para conquistar · Dos subarreglos independientes

1. Elegir un valor pivote.
2. Particionar el intervalo de modo que los valores pequeños queden a la izquierda.
3. Ordenar recursivamente ambos intervalos.
4. Terminar con cero o un elemento; usar trabajo serial para intervalos pequeños.



Usamos partición de Hoare: p es una frontera; no se supone que el pivote quede en p . Los intervalos no se solapan.

Por qué usar tareas para la recursión

Un equipo de hilos ejecuta muchas unidades de trabajo

Crear trabajo con task

El hilo genera una tarea. Puede ejecutarla de inmediato o dejarla disponible para otro hilo del mismo equipo.

Por qué usar tareas para la recursión

Un equipo de hilos ejecuta muchas unidades de trabajo

Crear trabajo con task

El hilo genera una tarea. Puede ejecutarla de inmediato o dejarla disponible para otro hilo del mismo equipo.

Coordinar con taskwait

La tarea actual espera a sus hijas ya creadas. En quicksort, cada hija espera a sus propias hijas antes de terminar.

Por qué usar tareas para la recursión

Un equipo de hilos ejecuta muchas unidades de trabajo

Crear trabajo con task

El hilo genera una tarea. Puede ejecutarla de inmediato o dejarla disponible para otro hilo del mismo equipo.

Coordinar con taskwait

La tarea actual espera a sus hijas ya creadas. En quicksort, cada hija espera a sus propias hijas antes de terminar.

- Crear una sola región `parallel` al inicio.

Por qué usar tareas para la recursión

Un equipo de hilos ejecuta muchas unidades de trabajo

Crear trabajo con `task`

El hilo genera una tarea. Puede ejecutarla de inmediato o dejarla disponible para otro hilo del mismo equipo.

Coordinar con `taskwait`

La tarea actual espera a sus hijas ya creadas. En quicksort, cada hija espera a sus propias hijas antes de terminar.

- Crear una sola región `parallel` al inicio.
- Usar `single` para iniciar una única recursión.

Por qué usar tareas para la recursión

Un equipo de hilos ejecuta muchas unidades de trabajo

Crear trabajo con `task`

El hilo genera una tarea. Puede ejecutarla de inmediato o dejarla disponible para otro hilo del mismo equipo.

Coordinar con `taskwait`

La tarea actual espera a sus hijas ya creadas. En quicksort, cada hija espera a sus propias hijas antes de terminar.

- Crear una sola región `parallel` al inicio.
- Usar `single` para iniciar una única recursión.
- Copiar límites y puntero con `firstprivate`.

Por qué usar tareas para la recursión

Un equipo de hilos ejecuta muchas unidades de trabajo

Crear trabajo con `task`

El hilo genera una tarea. Puede ejecutarla de inmediato o dejarla disponible para otro hilo del mismo equipo.

Coordinar con `taskwait`

La tarea actual espera a sus hijas ya creadas. En quicksort, cada hija espera a sus propias hijas antes de terminar.

El original crea `parallel sections` en cada llamada: puede serializar niveles internos o crear demasiados equipos. Las tareas evitan ese patrón.

- Crear una sola región `parallel` al inicio.
- Usar `single` para iniciar una única recursión.
- Copiar límites y puntero con `firstprivate`.
- Usar un umbral para no generar tareas diminutas.

Quicksort con task y taskwait

Intermedio · Núcleo recursivo

```
static void quicksort(int *a, int lo, int hi) {  
    if (lo >= hi) return;  
    if (hi - lo + 1 <= UMBRAL) {  
        qsort(a + lo, (size_t)(hi - lo + 1), sizeof(*a), comparar);  
        return;  
    }  
    int p = particionar(a, lo, hi);  
    #pragma omp task default(none) firstprivate(a,lo,p)  
    { quicksort(a, lo, p); }  
    #pragma omp task default(none) firstprivate(a,p,hi)  
    { quicksort(a, p + 1, hi); }  
    #pragma omp taskwait  
}
```

Programa completo: `examples/09_quicksort_tasks.c`

Quicksort con task y taskwait

Intermedio · Núcleo recursivo

```
static void quicksort(int *a, int lo, int hi) {  
    if (lo >= hi) return;  
    if (hi - lo + 1 <= UMBRAL) {  
        qsort(a + lo, (size_t)(hi - lo + 1), sizeof(*a), comparar);  
        return;  
    }  
    int p = particionar(a, lo, hi);  
    #pragma omp task default(none) firstprivate(a,lo,p)  
    { quicksort(a, lo, p); }  
    #pragma omp task default(none) firstprivate(a,p,hi)  
    { quicksort(a, p + 1, hi); }  
    #pragma omp taskwait  
}
```

UMBRAL=2048: resolver segmentos pequeños con qsort. Las tareas comparten los datos apuntados por a, pero escriben intervalos disjuntos.

Programa completo: `examples/09_quicksort_tasks.c`

Cómo elegir una construcción de OpenMP

Reconocer la forma del trabajo

Problema	Herramienta inicial	Verificación clave
Iteraciones independientes	<code>parallel for</code>	Cada salida tiene un escritor.

Cómo elegir una construcción de OpenMP

Reconocer la forma del trabajo

Problema	Herramienta inicial	Verificación clave
Iteraciones independientes	<code>parallel for</code>	Cada salida tiene un escritor.
Suma, conteo, máximo	<code>reduction</code>	Operación e identidad correctas.

Cómo elegir una construcción de OpenMP

Reconocer la forma del trabajo

Problema	Herramienta inicial	Verificación clave
Iteraciones independientes	<code>parallel for</code>	Cada salida tiene un escritor.
Suma, conteo, máximo	<code>reduction</code>	Operación e identidad correctas.
Pocos bloques diferentes	<code>sections</code>	Cada bloque produce su salida.

Cómo elegir una construcción de OpenMP

Reconocer la forma del trabajo

Problema	Herramienta inicial	Verificación clave
Iteraciones independientes	<code>parallel for</code>	Cada salida tiene un escritor.
Suma, conteo, máximo	<code>reduction</code>	Operación e identidad correctas.
Pocos bloques diferentes	<code>sections</code>	Cada bloque produce su salida.
Bucles rectangulares anidados	<code>collapse(2)</code>	No hay dependencias entre pares.

Cómo elegir una construcción de OpenMP

Reconocer la forma del trabajo

Problema	Herramienta inicial	Verificación clave
Iteraciones independientes	<code>parallel for</code>	Cada salida tiene un escritor.
Suma, conteo, máximo	<code>reduction</code>	Operación e identidad correctas.
Pocos bloques diferentes	<code>sections</code>	Cada bloque produce su salida.
Bucles rectangulares anidados	<code>collapse(2)</code>	No hay dependencias entre pares.
Recursión o trabajo irregular	<code>task</code>	Vida de datos y espera de tareas.

Cómo elegir una construcción de OpenMP

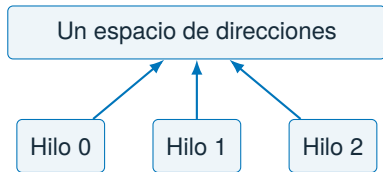
Reconocer la forma del trabajo

Problema	Herramienta inicial	Verificación clave
Iteraciones independientes	<code>parallel for</code>	Cada salida tiene un escritor.
Suma, conteo, máximo	<code>reduction</code>	Operación e identidad correctas.
Pocos bloques diferentes	<code>sections</code>	Cada bloque produce su salida.
Bucles rectangulares anidados	<code>collapse(2)</code>	No hay dependencias entre pares.
Recursión o trabajo irregular	<code>task</code>	Vida de datos y espera de tareas.
Acumulación en muchas celdas	Datos locales + fusión	La fusión protege todas las escrituras.

MPI: cuando cada proceso tiene su memoria

Message Passing Interface

Memoria compartida · OpenMP

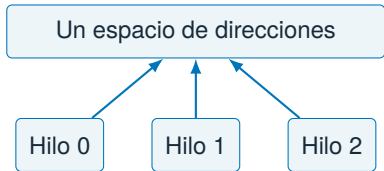


Los hilos pueden acceder al mismo arreglo. Hay que coordinar accesos en conflicto.

MPI: cuando cada proceso tiene su memoria

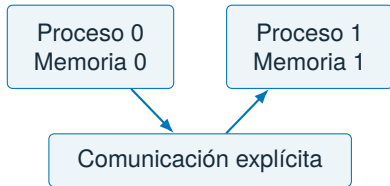
Message Passing Interface

Memoria compartida · OpenMP



Los hilos pueden acceder al mismo arreglo. Hay que coordinar accesos en conflicto.

Memoria distribuida · MPI



Los procesos intercambian datos mediante llamadas MPI, en una máquina o entre nodos.

Qué aporta el paso de mensajes

Ventajas y costos que debemos considerar

Ventajas

- Se adapta a nodos conectados por una red.

Costos y límites

- La red añade latencia y limita el ancho de banda.

Qué aporta el paso de mensajes

Ventajas y costos que debemos considerar

Ventajas

- Se adapta a nodos conectados por una red.
- Expresa particiones de datos de forma explícita.

Costos y límites

- La red añade latencia y limita el ancho de banda.
- Hay que equilibrar trabajo y comunicaciones.

Qué aporta el paso de mensajes

Ventajas y costos que debemos considerar

Ventajas

- Se adapta a nodos conectados por una red.
- Expresa particiones de datos de forma explícita.
- Hace visible quién posee y comunica cada dato.

Costos y límites

- La red añade latencia y limita el ancho de banda.
- Hay que equilibrar trabajo y comunicaciones.
- Un mensaje sin recepción compatible puede bloquear el programa.

Qué aporta el paso de mensajes

Ventajas y costos que debemos considerar

Ventajas

- Se adapta a nodos conectados por una red.
- Expresa particiones de datos de forma explícita.
- Hace visible quién posee y comunica cada dato.
- Permite usar memoria y cómputo de varios nodos.

Costos y límites

- La red añade latencia y limita el ancho de banda.
- Hay que equilibrar trabajo y comunicaciones.
- Un mensaje sin recepción compatible puede bloquear el programa.
- Espacios separados no eliminan errores de sincronización.

MPI es una especificación de una interfaz

Las implementaciones proporcionan bibliotecas y herramientas

- No es un lenguaje: se usa desde programas C, C++ o Fortran mediante sus interfaces disponibles.

MPI es una especificación de una interfaz

Las implementaciones proporcionan bibliotecas y herramientas

- No es un lenguaje: se usa desde programas C, C++ o Fortran mediante sus interfaces disponibles.
- El estándar define las operaciones y su comportamiento. Open MPI y MPICH son implementaciones.

MPI es una especificación de una interfaz

Las implementaciones proporcionan bibliotecas y herramientas

- No es un lenguaje: se usa desde programas C, C++ o Fortran mediante sus interfaces disponibles.
- El estándar define las operaciones y su comportamiento. Open MPI y MPICH son implementaciones.
- `mpicc` facilita compilar y enlazar un programa C con la biblioteca MPI instalada.

MPI es una especificación de una interfaz

Las implementaciones proporcionan bibliotecas y herramientas

- No es un lenguaje: se usa desde programas C, C++ o Fortran mediante sus interfaces disponibles.
- El estándar define las operaciones y su comportamiento. Open MPI y MPICH son implementaciones.
- `mpicc` facilita compilar y enlazar un programa C con la biblioteca MPI instalada.
- `mpirun -np 4` inicia cuatro procesos en el entorno configurado.

MPI es una especificación de una interfaz

Las implementaciones proporcionan bibliotecas y herramientas

- No es un lenguaje: se usa desde programas C, C++ o Fortran mediante sus interfaces disponibles.
- El estándar define las operaciones y su comportamiento. Open MPI y MPICH son implementaciones.
- `mpicc` facilita compilar y enlazar un programa C con la biblioteca MPI instalada.
- `mpirun -np 4` inicia cuatro procesos en el entorno configurado.

Modelo SPMD

Los procesos ejecutan el mismo programa, pero pueden trabajar con datos distintos y tomar ramas distintas según su identificador o *rank*.

Aquí estudiamos comunicación de dos lados y colectivas. MPI también ofrece otras capacidades, como operaciones de acceso remoto a memoria.

Vocabulario y funciones mínimas

Proceso, rank y comunicador

Comunicador

Define un grupo y un contexto de comunicación. Usaremos `MPI_COMM_WORLD`.

Vocabulario y funciones mínimas

Proceso, rank y comunicador

Comunicador

Define un grupo y un contexto de comunicación. Usaremos `MPI_COMM_WORLD`.

Rank y tamaño

Con P procesos, los ranks van de 0 a $P - 1$. Son relativos al comunicador; no identifican núcleos físicos.

Vocabulario y funciones mínimas

Proceso, rank y comunicador

Comunicador

Define un grupo y un contexto de comunicación. Usaremos `MPI_COMM_WORLD`.

Rank y tamaño

Con P procesos, los ranks van de 0 a $P - 1$. Son relativos al comunicador; no identifican núcleos físicos.

Función	Propósito
<code>MPI_Init</code>	Inicializa el entorno MPI.

Vocabulario y funciones mínimas

Proceso, rank y comunicador

Comunicador

Define un grupo y un contexto de comunicación. Usaremos `MPI_COMM_WORLD`.

Rank y tamaño

Con P procesos, los ranks van de 0 a $P - 1$. Son relativos al comunicador; no identifican núcleos físicos.

Función	Propósito
<code>MPI_Init</code>	Inicializa el entorno MPI.
<code>MPI_Comm_size</code>	Número de procesos del grupo.
<code>MPI_Comm_rank</code>	Rank del proceso que llama.

Vocabulario y funciones mínimas

Proceso, rank y comunicador

Comunicador

Define un grupo y un contexto de comunicación. Usaremos `MPI_COMM_WORLD`.

Rank y tamaño

Con P procesos, los ranks van de 0 a $P - 1$. Son relativos al comunicador; no identifican núcleos físicos.

Función	Propósito
<code>MPI_Init</code>	Inicializa el entorno MPI.
<code>MPI_Comm_size</code>	Número de procesos del grupo.
<code>MPI_Comm_rank</code>	Rank del proceso que llama.
<code>MPI_Send</code>	Envía un mensaje.
<code>MPI_Recv</code>	Recibe un mensaje compatible.

Vocabulario y funciones mínimas

Proceso, rank y comunicador

Comunicador

Define un grupo y un contexto de comunicación. Usaremos `MPI_COMM_WORLD`.

Rank y tamaño

Con P procesos, los ranks van de 0 a $P - 1$. Son relativos al comunicador; no identifican núcleos físicos.

Función	Propósito
<code>MPI_Init</code>	Inicializa el entorno MPI.
<code>MPI_Comm_size</code>	Número de procesos del grupo.
<code>MPI_Comm_rank</code>	Rank del proceso que llama.
<code>MPI_Send</code>	Envía un mensaje.
<code>MPI_Recv</code>	Recibe un mensaje compatible.
<code>MPI_Finalize</code>	Finaliza el uso de MPI.

Primer programa MPI: hola mundo

Un saludo por proceso

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size, longitud;
    char nombre[MPI_MAX_PROCESSOR_NAME];
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Get_processor_name(nombre, &longitud);
    printf("Hola desde rank %d de %d en %.*s\n",
           rank, size, longitud, nombre);
    MPI_Finalize();
    return 0;
}
```

Primer programa MPI: hola mundo

Un saludo por proceso

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size, longitud;
    char nombre[MPI_MAX_PROCESSOR_NAME];
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Get_processor_name(nombre, &longitud);
    printf("Hola desde rank %d de %d en %.*s\n",
           rank, size, longitud, nombre);
    MPI_Finalize();
    return 0;
}
```

Con cuatro procesos aparecen cuatro saludos con ranks distintos.

Primer programa MPI: hola mundo

Un saludo por proceso

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size, longitud;
    char nombre[MPI_MAX_PROCESSOR_NAME];
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Get_processor_name(nombre, &longitud);
    printf("Hola desde rank %d de %d en %.*s\n",
           rank, size, longitud, nombre);
    MPI_Finalize();
    return 0;
}
```

Con cuatro procesos aparecen cuatro saludos con ranks distintos.

Varios procesos pueden mostrar el mismo nombre de máquina.

El orden de impresión no está garantizado.

Primer programa MPI: hola mundo

Un saludo por proceso

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size, longitud;
    char nombre[MPI_MAX_PROCESSOR_NAME];
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Get_processor_name(nombre, &longitud);
    printf("Hola desde rank %d de %d en %.*s\n",
           rank, size, longitud, nombre);
    MPI_Finalize();
    return 0;
}
```

```
mpicc -O2 examples/mpi_01_hola.c -o mpi_hola
mpirun -np 4 ./mpi_hola
```

Con cuatro procesos aparecen cuatro saludos con ranks distintos.

Varios procesos pueden mostrar el mismo nombre de máquina.

El orden de impresión no está garantizado.

Anatomía de un mensaje

Comunicación punto a punto de dos lados

```
MPI_Send(buffer, count, datatype, dest, tag, comm);  
MPI_Recv(buffer, capacity, datatype, source, tag, comm, status);
```

Anatomía de un mensaje

Comunicación punto a punto de dos lados

```
MPI_Send(buffer, count, datatype, dest, tag, comm);  
MPI_Recv(buffer, capacity, datatype, source, tag, comm, status);
```

- buffer: dirección de los datos locales.
- dest/source: rank del otro proceso.

Anatomía de un mensaje

Comunicación punto a punto de dos lados

```
MPI_Send(buffer, count, datatype, dest, tag, comm);  
MPI_Recv(buffer, capacity, datatype, source, tag, comm, status);
```

- `buffer`: dirección de los datos locales.
- `count`: cantidad de elementos del tipo indicado, no necesariamente bytes.
- `dest/source`: rank del otro proceso.
- `tag`: etiqueta entera para distinguir mensajes.

Anatomía de un mensaje

Comunicación punto a punto de dos lados

```
MPI_Send(buffer, count, datatype, dest, tag, comm);  
MPI_Recv(buffer, capacity, datatype, source, tag, comm, status);
```

- `buffer`: dirección de los datos locales.
- `count`: cantidad de elementos del tipo indicado, no necesariamente bytes.
- `capacity`: espacio disponible en la recepción.
- `dest/source`: rank del otro proceso.
- `tag`: etiqueta entera para distinguir mensajes.
- `comm`: contexto de comunicación; `status`: información del mensaje recibido.

Anatomía de un mensaje

Comunicación punto a punto de dos lados

```
MPI_Send(buffer, count, datatype, dest, tag, comm);  
MPI_Recv(buffer, capacity, datatype, source, tag, comm, status);
```

- **buffer**: dirección de los datos locales.
- **count**: cantidad de elementos del tipo indicado, no necesariamente bytes.
- **capacity**: espacio disponible en la recepción.
- **dest/source**: rank del otro proceso.
- **tag**: etiqueta entera para distinguir mensajes.
- **comm**: contexto de comunicación; **status**: información del mensaje recibido.

Compatibilidad

La recepción debe coincidir en comunicador, fuente y etiqueta, y tener tipos compatibles y espacio suficiente. Usaremos fuentes y etiquetas explícitas.

Enviar un entero del rank 0 al rank 1

Ejemplo mínimo con exactamente dos procesos

```
int valor = 0;
if (rank == 0) {
    valor = 42;
    MPI_Send(&valor, 1, MPI_INT, 1, 7, MPI_COMM_WORLD);
} else if (rank == 1) {
    MPI_Recv(&valor, 1, MPI_INT, 0, 7,
            MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Rank 1 recibio %d\n", valor);
}
```

Programa completo: `examples/mpi_02_mensaje.c`

Enviar un entero del rank 0 al rank 1

Ejemplo mínimo con exactamente dos procesos

```
int valor = 0;
if (rank == 0) {
    valor = 42;
    MPI_Send(&valor, 1, MPI_INT, 1, 7, MPI_COMM_WORLD);
} else if (rank == 1) {
    MPI_Recv(&valor, 1, MPI_INT, 0, 7,
            MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Rank 1 recibio %d\n", valor);
}
```

```
make mpi
mpirun -np 2 ./build/examples/mpi_02_mensaje
# Rank 1 recibio 42
```

Programa completo: `examples/mpi_02_mensaje.c`

Enviar un entero del rank 0 al rank 1

Ejemplo mínimo con exactamente dos procesos

```
int valor = 0;
if (rank == 0) {
    valor = 42;
    MPI_Send(&valor, 1, MPI_INT, 1, 7, MPI_COMM_WORLD);
} else if (rank == 1) {
    MPI_Recv(&valor, 1, MPI_INT, 0, 7,
            MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Rank 1 recibio %d\n", valor);
}
```

```
make mpi
mpirun -np 2 ./build/examples/mpi_02_mensaje
# Rank 1 recibio 42
```

Cada proceso tiene su propia variable `valor`; el envío transfiere su contenido. El programa completo verifica que existan exactamente dos procesos.

Programa completo: `examples/mpi_02_mensaje.c`

El orden de las comunicaciones importa

Evitar esperas circulares

Patrón que puede bloquearse

Proceso 0: enviar a 1; luego recibir de 1.

Proceso 1: enviar a 0; luego recibir de 0.

El orden de las comunicaciones importa

Evitar esperas circulares

Patrón que puede bloquearse

Proceso 0: enviar a 1; luego recibir de 1.

Proceso 1: enviar a 0; luego recibir de 0.

MPI_Send puede esperar una recepción compatible. Que funcione con mensajes pequeños no demuestra que el patrón sea correcto.

El orden de las comunicaciones importa

Evitar esperas circulares

Patrón que puede bloquearse

Proceso 0: enviar a 1; luego recibir de 1.

Proceso 1: enviar a 0; luego recibir de 0.

`MPI_Send` puede esperar una recepción compatible. Que funcione con mensajes pequeños no demuestra que el patrón sea correcto.

Alternativas

Ordenar: un proceso envía primero y el otro recibe primero.

Usar `MPI_Sendrecv` para un intercambio.

Publicar recepciones y envíos no bloqueantes y esperar su finalización.

El orden de las comunicaciones importa

Evitar esperas circulares

Patrón que puede bloquearse

Proceso 0: enviar a 1; luego recibir de 1.

Proceso 1: enviar a 0; luego recibir de 0.

`MPI_Send` puede esperar una recepción compatible. Que funcione con mensajes pequeños no demuestra que el patrón sea correcto.

Precisión: al retornar `MPI_Send`, el buffer de envío puede reutilizarse; eso no implica que el receptor ya haya terminado de procesar los datos.

Alternativas

Ordenar: un proceso envía primero y el otro recibe primero.

Usar `MPI_Sendrecv` para un intercambio.

Publicar recepciones y envíos no bloqueantes y esperar su finalización.

Comunicación colectiva

Todos los procesos del comunicador participan

Operación	Esquema	Propósito
MPI_Bcast	$x \rightarrow (x, x, x)$	Copiar datos desde una raíz a todos.

Comunicación colectiva

Todos los procesos del comunicador participan

Operación	Esquema	Propósito
MPI_Bcast	$x \rightarrow (x, x, x)$	Copiar datos desde una raíz a todos.
MPI_Scatter	$(a, b, c) \rightarrow a \mid b \mid c$	Distribuir bloques desde la raíz.

Comunicación colectiva

Todos los procesos del comunicador participan

Operación	Esquema	Propósito
MPI_Bcast	$x \rightarrow (x, x, x)$	Copiar datos desde una raíz a todos.
MPI_Scatter	$(a, b, c) \rightarrow a \mid b \mid c$	Distribuir bloques desde la raíz.
MPI_Gather	$a \mid b \mid c \rightarrow (a, b, c)$	Reunir bloques en la raíz.

Comunicación colectiva

Todos los procesos del comunicador participan

Operación	Esquema	Propósito
MPI_Bcast	$x \rightarrow (x, x, x)$	Copiar datos desde una raíz a todos.
MPI_Scatter	$(a, b, c) \rightarrow a \mid b \mid c$	Distribuir bloques desde la raíz.
MPI_Gather	$a \mid b \mid c \rightarrow (a, b, c)$	Reunir bloques en la raíz.
MPI_Reduce	$a \mid b \mid c \rightarrow a + b + c$	Combinar y entregar en la raíz.

Comunicación colectiva

Todos los procesos del comunicador participan

Operación	Esquema	Propósito
MPI_Bcast	$x \rightarrow (x, x, x)$	Copiar datos desde una raíz a todos.
MPI_Scatter	$(a, b, c) \rightarrow a \mid b \mid c$	Distribuir bloques desde la raíz.
MPI_Gather	$a \mid b \mid c \rightarrow (a, b, c)$	Reunir bloques en la raíz.
MPI_Reduce	$a \mid b \mid c \rightarrow a + b + c$	Combinar y entregar en la raíz.
MPI_Allreduce	$a \mid b \mid c \rightarrow (s, s, s)$	Combinar y entregar a todos.

Comunicación colectiva

Todos los procesos del comunicador participan

Operación	Esquema	Propósito
MPI_Bcast	$x \rightarrow (x, x, x)$	Copiar datos desde una raíz a todos.
MPI_Scatter	$(a, b, c) \rightarrow a \mid b \mid c$	Distribuir bloques desde la raíz.
MPI_Gather	$a \mid b \mid c \rightarrow (a, b, c)$	Reunir bloques en la raíz.
MPI_Reduce	$a \mid b \mid c \rightarrow a + b + c$	Combinar y entregar en la raíz.
MPI_Allreduce	$a \mid b \mid c \rightarrow (s, s, s)$	Combinar y entregar a todos.
MPI_Barrier	Todos esperan	Sincronizar al grupo.

Comunicación colectiva

Todos los procesos del comunicador participan

Operación	Esquema	Propósito
MPI_Bcast	$x \rightarrow (x, x, x)$	Copiar datos desde una raíz a todos.
MPI_Scatter	$(a, b, c) \rightarrow a \mid b \mid c$	Distribuir bloques desde la raíz.
MPI_Gather	$a \mid b \mid c \rightarrow (a, b, c)$	Reunir bloques en la raíz.
MPI_Reduce	$a \mid b \mid c \rightarrow a + b + c$	Combinar y entregar en la raíz.
MPI_Allreduce	$a \mid b \mid c \rightarrow (s, s, s)$	Combinar y entregar a todos.
MPI_Barrier	Todos esperan	Sincronizar al grupo.

Reglas para nuestros ejemplos con MPI_COMM_WORLD

Todos llaman a las colectivas en el mismo orden y con argumentos compatibles. Una colectiva como MPI_Bcast no sustituye automáticamente una barrera.

La suma de 1 a 10, ahora con MPI

Repartir el cálculo local y combinar con MPI_Reduce

```
int n = 10, local = 0, total = 0;
/* Distribucion ciclica: cubre todos los elementos, aun si n % size != 0. */
for (int i = rank; i < n; i += size)
    local += i + 1;
MPI_Reduce(&local, &total, 1, MPI_INT, MPI_SUM,
           0, MPI_COMM_WORLD);
if (rank == 0) printf("Suma global = %d\n", total);
```

Programa completo: `examples/mpi_03_reduce.c`

La suma de 1 a 10, ahora con MPI

Repartir el cálculo local y combinar con MPI_Reduce

```
int n = 10, local = 0, total = 0;
/* Distribucion ciclica: cubre todos los elementos, aun si n % size != 0. */
for (int i = rank; i < n; i += size)
    local += i + 1;
MPI_Reduce(&local, &total, 1, MPI_INT, MPI_SUM,
           0, MPI_COMM_WORLD);
if (rank == 0) printf("Suma global = %d\n", total);
```

Cuatro procesos: sumas locales

Rank 0: $1 + 5 + 9 = 15$.

Rank 1: $2 + 6 + 10 = 18$.

Rank 2: $3 + 7 = 10$.

Rank 3: $4 + 8 = 12$.

Programa completo: `examples/mpi_03_reduce.c`

La suma de 1 a 10, ahora con MPI

Repartir el cálculo local y combinar con MPI_Reduce

```
int n = 10, local = 0, total = 0;
/* Distribucion ciclica: cubre todos los elementos, aun si n % size != 0. */
for (int i = rank; i < n; i += size)
    local += i + 1;
MPI_Reduce(&local, &total, 1, MPI_INT, MPI_SUM,
           0, MPI_COMM_WORLD);
if (rank == 0) printf("Suma global = %d\n", total);
```

Cuatro procesos: sumas locales

Rank 0: $1 + 5 + 9 = 15$.

Rank 1: $2 + 6 + 10 = 18$.

Rank 2: $3 + 7 = 10$.

Rank 3: $4 + 8 = 12$.

Resultado en la raíz

$15 + 18 + 10 + 12 = 55$.

Todos llaman a MPI_Reduce; solo el rank 0 usa total. Los valores se generan localmente: no hace falta enviarlos antes.

Programa completo: `examples/mpi_03_reduce.c`

Material de práctica y notebooks

Enlaces conservados del PDF original

- [OpenMP.ipynb](#): repaso de regiones y trabajo compartido.

Material de práctica y notebooks

Enlaces conservados del PDF original

- [OpenMP.ipynb](#): repaso de regiones y trabajo compartido.
- [OMP2.ipynb](#): ejemplos de OpenMP II.

Material de práctica y notebooks

Enlaces conservados del PDF original

- [OpenMP.ipynb](#): repaso de regiones y trabajo compartido.
- [OMP2.ipynb](#): ejemplos de OpenMP II.
- [MPI_I.ipynb](#): introducción a MPI.